

Programmieren lernen
mit Python und MicroPython



Arbeitsblätter zum Kurs

Inhalt

Programmzeilen mit der IDLE Shell testen	3
Mit der IDLE Shell rechnen.....	4
Mit dem IDLE Editor ein neues Programm schreiben	5
Mit dem IDLE Editor ein vorhandenes Programm bearbeiten.....	6
Eine Liste speichert viele änderbare Elemente	7
Ein Tupel speichert viele nicht änderbare Elemente	8
Ein Dictionary speichert Paare von Schlüssel und Wert	9
Der Computer fragt	10
Dein Wörterbuch lernt	11
Der Computer unterscheidet zwischen "wahr" und "falsch"	12
Der Computer unterscheidet Fälle.....	13
Solange die Bedingung wahr ist, ausführen	14
Der Computer errät die Zahl	16
Funktionen in Python.....	17
Funktionen haben Input und Output: Ganzzahlen	18
Funktionen haben Input und Output: Tupel, Liste.....	19
Globale und lokale Variable in der Funktion	20
Funktion konstruieren: Bau den Satz	21
Funktion konstruieren: GBP in EUR.....	23
Funktion konstruieren: Kosten für Tennisunterricht.....	25
Klassen haben Eigenschaften und Methoden.....	28
Baue deine eigene Klasse.....	29
Die Eltern-Klasse vererbt – die Kind-Klasse erbt.....	30
Baue Eltern-Klasse und Kind-Klassen	31
Roboter im Irrgarten: Wände bauen.....	33
Roboter im Irrgarten: Wände ablegen	34
Roboter im Irrgarten: Wände aus Datei lesen.....	35
Roboter im Irrgarten: Schildkröte bewegen.....	36
Roboter im Irrgarten: Schauen, gehen, drehen	37
Roboter im Irrgarten: Rechte-Hand-Methode	39
Roboter im Irrgarten: Pledge-Algorithmus.....	40
Auto im Gegenverkehr: Auto steuern	42
Auto im Gegenverkehr: Gegenverkehr kommt	44
Auto im Gegenverkehr: Zusammenstoß melden	45

Auto im Gegenverkehr: Gegenverkehr wird schneller	47
Auto im Gegenverkehr: Straße und Sound	48
Ein Dinosaurier überquert die Straße	49
MicroPython und ESP32: Hardware und Software	50
MicroPython: Displays, Töne, Aktoren	52
MicroPython: Sensoren.....	53
MicroPython: Einparkhilfe	54
Quellen.....	55

Programmzeilen mit der IDLE Shell testen

Gib die Programmzeilen nach dem Prompt (>>>) ein. Ist das Ergebnis OK oder bekommst du eine Fehlermeldung?

Tipp: Mit ALT + p kannst du die letzte Zeile wiederholen.

Zeichenketten (strings) testen

Programmzeile	OK	Fehler
Hallo Welt		
"Hallo Welt"		
"Hallo Welt'		
'Hallo Welt'		
print("Hallo Welt")		
pirnt("Hallo Welt")		

Zeichenketten in Variablen schreiben

Programmzeile	OK	Fehler
nachricht = Hallo Welt		
nachricht = "Hallo Welt"		
nachricht		
print(nachricht)		
len(nachricht)		
type(nachricht)		
nachricht.upper()		

Zahlen (integer und float) testen

Programmzeile	OK	Fehler
3		
3.14		
3,14		
print(3)		
print(3.14)		
print(3,14)		

Zahlen in Variablen schreiben

Programmzeile	OK	Fehler
zahl = 3		
zahl = 3.14		
zahl		
print(zahl)		
len(zahl)		
type(zahl)		
zahl.upper()		

Die Farbe kennzeichnet einen Text mit besonderer Bedeutung

Farbe	Bedeutung
rot	
grün	
violett	

Mit der IDLE Shell rechnen

Gib die Programmzeilen nach dem Prompt (>>>) ein. Ist das Ergebnis OK oder bekommst du eine Fehlermeldung?

Rechnen mit Zeichenketten

Programmzeile	OK	Fehler
"Hallo" + "Welt"		
"Hallo" + " " + "Welt"		
"Hallo " + "Welt"		
"Hallo " - "Welt"		
print("Hallo " + "Welt")		
3 * "Hallo Welt"		
3 * "Hallo Welt "		
3 / "Hallo Welt "		
print(3 * "Hallo Welt ")		
nachricht = "Hallo Welt "		
print(3 * nachricht)		

Rechnen mit Zahlen

Programmzeile	OK	Fehler
3 + 4		
3 - 4		
3 * 4		
3/4		
30//4		
30%4		
ergebnis = 3 * 4		
ergebnis/5		
ergebnis//5		
print(ergebnis/5)		

Punktrechnung vor Strichrechnung

Programmzeile	OK	Fehler
3 + 4 * 2		
(3 + 4) * 2		
12 - 3 / 2		
(12 - 3) / 2		
12 - 3 // 2		
(12 - 3) // 2		

Zahl in Zeichenkette umwandeln – Zeichenkette in Zahl umwandeln

Programmzeile	OK	Fehler
zahl = 3		
str(zahl)		
zahl = 3.14		
str(zahl)		
zeichenkette = "3.14"		
int(zeichenkette)		
float(zeichenkette)		

Mit dem IDLE Editor ein neues Programm schreiben



Mein erstes Programm!

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Gib die **Programmzeilen** im Editor-Fenster ein. Die Farbe dort kennzeichnet einen Text mit besonderer Bedeutung.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "mein_erstes_programm" im Ordner Python/03_Rechnen_mit_Zeichenketten_und_Zahlen
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- Notiere die print-Ausgaben in der Tabelle.

Rechnen

Programmzeile	print-Ausgabe
# Das ist ein Kommentar	
# Rechnen	
nachricht = "Hallo Welt "	
print(3 * nachricht)	
ergebnis = 3 * 4	
print(ergebnis/5)	

Umwandeln

Programmzeile	print-Ausgabe
# Umwandeln	
zahl = 3.14	
print(zahl)	
print(str(zahl))	
zeichenkette = "3.14 "	
print(zeichenkette)	
print(3 * zeichenkette)	
print(3 * float(zeichenkette))	

Variablen ausgeben

Programmzeile	print-Ausgabe
# Variablen ausgeben	
print("nachricht =", nachricht)	
print("ergebnis =", ergebnis)	
print("zahl =", zahl)	
print("zeichenkette =", zeichenkette)	

Mit dem IDLE Editor ein vorhandenes Programm bearbeiten



Zeichne eine Ziffer!

Die Ziffern auf einem Kassenzettel sind aus Punkten zusammengesetzt. Mit 5x7 Punkten können die Ziffern 0 bis 9 gut lesbar dargestellt werden.

	•	•	•	
•				•
•				•
•				•
•				•
•				•
	•	•	•	

- Mit dem Menüpunkt **File/ Open ...** öffnest du die Datei "Zeichne_eine_Ziffer.py" im Ordner Python/03_Rechnen_mit_Zeichenketten_und_Zahlen.
- In "Zeichne_eine_Ziffer.py" findest du Kommentare und eine Programmzeile. Zeichne mit print-Befehlen und dem Zeichen • eine Ziffer in ein 5x7 Raster:
 - a) Zeichne eine Null
 - b) Zeichne eine Acht
 - c) Zeichne eine Eins
 - d) Zeichne eine Ziffer deiner Wahl
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- Teste dein Programm nach jeder Aufgabe.

Tipp: Du kannst die Programmzeile mit dem Zeichen • kopieren und einfügen.

Eine Liste speichert viele änderbare Elemente

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Merke: Der Index steht in eckigen Klammern. Der Index beginnt mit Null!

Notiere die print-Ausgaben in der Tabelle.

Eine Liste anlegen – mit eckigen Klammern!

Programmzeile	print-Ausgabe
<code>vornamen = ["Axel", "Elke", "Martin"]</code>	
<code>print(vornamen)</code>	
<code>print(vornamen[0])</code>	
<code>print(vornamen[0:2])</code>	
<code>print(vornamen[-1])</code>	
<code>vornamen[2] = "Fritz"</code>	
<code>print(vornamen)</code>	

Eine Liste erweitern

Programmzeile	print-Ausgabe
<code>vornamen = vornamen + ["Heike", "Sabine"]</code>	
<code>print(vornamen)</code>	
<code>vornamen += ["Markus"]</code>	
<code>print(vornamen)</code>	

Eine leere Liste anlegen und füllen

Programmzeile	print-Ausgabe
<code>buchstaben = []</code>	
<code>buchstaben.append("a")</code>	
<code>print(buchstaben)</code>	
<code>buchstaben.append("b")</code>	
<code>print(buchstaben)</code>	

Elemente einer Liste löschen

Programmzeile	print-Ausgabe
<code>print(vornamen)</code>	
<code>vornamen.remove("Heike")</code>	
<code>print(vornamen)</code>	
<code>del vornamen[0]</code>	
<code>print(vornamen)</code>	
<code>del vornamen</code>	
<code>print(vornamen)</code>	

Ein zufälliges Element aus einer Liste auswählen

Programmzeile	print-Ausgabe
<code>import random</code>	
<code>handzeichen = ["Schere", "Stein", "Papier"]</code>	
<code>print(random.choice(handzeichen))</code>	

Ein Tupel speichert viele nicht änderbare Elemente

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Merke: Der Index steht in eckigen Klammern. Der Index beginnt mit Null!

Notiere die print-Ausgaben in der Tabelle.

Ein Tupel anlegen – mit runden Klammern!

Programmzeile	print-Ausgabe
punkt = (-10, 5, 7)	
print(punkt)	
print(punkt[0])	
print(punkt[0:2])	
print(punkt[-1])	
punkt[2] = 15	
punkt = (-10, 5, 15)	
print(punkt)	

Ein Element im Tupel suchen

Programmzeile	print-Ausgabe
print(punkt)	
print(punkt.count(7))	
print(punkt.index(7))	

Ein Dictionary speichert Paare von Schlüssel und Wert



Wörterbuch!

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Gib die **Programmzeilen** im Editor-Fenster ein.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "woerterbuch" im Ordner Python/04_Listen_und_Woerterbuecher
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- Notiere die print-Ausgaben in der Tabelle.

Ein leeres dictionary anlegen – mit geschweiften Klammern – und füllen

Programmzeile	print-Ausgabe
# Wörterbuch Englisch - Deutsch	
# Leeres dictionary anlegen	
englisch_deutsch = {}	
# dictionary füllen	
englisch_deutsch["cat"] = "Katze"	
englisch_deutsch["dog"] = "Hund"	
englisch_deutsch["cow"] = "Kuh"	
print(englisch_deutsch)	
print(englisch_deutsch["dog"])	
englisch_deutsch["sheep"] = "Schaf"	
print(englisch_deutsch)	

Schlüssel und Werte des dictionary ausgeben

Programmzeile	print-Ausgabe
# Schlüssel ausgeben	
print(englisch_deutsch.keys())	
# Werte ausgeben	
print(englisch_deutsch.values())	

Neues dictionary – in einer Zeile – anlegen

Wenn jeder Wert nur einmal im dictionary vorkommt, geht das mit folgender Programmzeile (Erläuterung später).

Programmzeile	print-Ausgabe
# Dictionary in einer Zeile anlegen	
in_my_town = {"house": Haus, "street": "Straße", "people": "Leute"}	
print(in_my_town)	
print(in_my_town["people"])	

Der Computer fragt ...



Summe ausgeben!

- Gib die **Programmzeilen** im Editor-Fenster ein.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "summe_ausgeben" im Ordner Python/05_Benutzereingaben
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- Notiere die print-Ausgaben in der Tabelle.

Der Computer erwartet Zahlen

Programmzeile	print-Ausgabe
# Summe von zwei Zahlen ausgeben	
# Benutzereingaben anfordern	
zahl1 = input("Gib die erste Zahl ein ")	
zahl2 = input("Gib die zweite Zahl ein ")	
# Strings in Dezimalzahlen umwandeln	
zahl1 = float(zahl1)	
zahl2 = float(zahl2)	
# Summe ausgeben	
print("Die Summe der Zahlen ist", zahl1 + zahl2)	

Der Computer erwartet Strings

Programmzeile	print-Ausgabe
# Summe von zwei Strings ausgeben	
# Benutzereingaben anfordern	
str1 = input("Gib den ersten String ein ")	
str2 = input("Gib den zweiten String ein ")	
# Summe ausgeben	
print("Die Summe der Strings ist", str1 + str2)	

Dein Wörterbuch lernt ...



Wörterbuch erweitern!

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Beginne mit einem Kommentar, z. B.
`# Das Dictionary englisch_deutsch soll erweitert werden`
- Nun schreibe die Kommentare und **entwicke** die Programmzeilen:
 - a) Lege das Dictionary `englisch_deutsch` an und fülle es mit 3 Paaren. Zeige das Dictionary.
 - b) Frage nach einem neuen Wort. Lass den Nutzer ein neues englisches Wort eingeben – das wird dein **Schlüssel**.
 - c) Frage nach der Übersetzung. Lass den Nutzer die passende deutsche Übersetzung eingeben – das wird dein **Wert**.
 - d) Erweitere dein Wörterbuch. Füge das neue Paar von Schlüssel und Wert zu deinem Dictionary hinzu.
 - e) Zeige das Ergebnis. Gib das komplette, erweiterte Wörterbuch aus.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "dictionary_erweitern" im Ordner `Python/05_Benutzereingaben`
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- So könnte es aussehen:

```
{'cat': 'Katze', 'dog': 'Hund', 'cow': 'Kuh'}  
Gib ein englisches Wort an: bird  
Gib das passende deutsche Wort an: Vogel  
{'cat': 'Katze', 'dog': 'Hund', 'cow': 'Kuh', 'bird': 'Vogel'}
```

- Knifflige **Zusatzfrage**: Was passiert wohl, wenn jemand ein englisches Wort eingibt, das schon in deinem Wörterbuch steht? Teste es und schreibe auf, was du beobachtest!

Bei der Formulierung dieser Aufgabe wurde Claude (Anthropic) unterstützend eingesetzt.

Der Computer unterscheidet zwischen "wahr" und "falsch"

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Notiere die print-Ausgaben in der Tabelle.

Bedingung mit Zahlen

Programmzeile	print-Ausgabe
<code>print(1 == 2)</code>	
<code>print(1 != 2)</code>	
<code>print(1 < 2)</code>	
<code>print(1 > 2)</code>	
<code>print(3 == 3)</code>	
<code>print(3 != 3)</code>	
<code>print(3 <= 3)</code>	
<code>print(3 >= 3)</code>	

Bedingung mit Strings

Programmzeile	print-Ausgabe
<code>print("Hallo" == "Welt")</code>	
<code>print("Hallo" != "Welt")</code>	
<code>print("Montag" == "Montag")</code>	
<code>print("Montag" != "Montag")</code>	

Bedingung mit range(stop)

Programmzeile	print-Ausgabe
<code>bereich = range(10)</code>	
<code>print(list(bereich))</code>	
<code>print(1 in bereich)</code>	
<code>print(10 in bereich)</code>	

Bedingung mit range(start, stop)

Programmzeile	print-Ausgabe
<code>bereich = range(2, 10)</code>	
<code>print(list(bereich))</code>	
<code>print(1 in bereich)</code>	
<code>print(9 in bereich)</code>	

Bedingung mit range(start, stop, step)

Programmzeile	print-Ausgabe
<code>bereich = range(0, 10, 2)</code>	
<code>print(list(bereich))</code>	
<code>print(3 in bereich)</code>	
<code>print(8 in bereich)</code>	

Der Computer unterscheidet Fälle

Wenn die Bedingung "wahr" ist, werden die Programmzeilen darunter ausgeführt

Gib die Programmzeilen nach dem Prompt (>>>) ein.

Notiere die print-Ausgaben in der Tabelle.

Eine Bedingung – zwei Fälle

Programmzeile	print-Ausgabe
wert = 5	
if wert < 10:	
print("wert ist kleiner als 10")	
print("Ich gehöre auch zu der Bedingung")	

Eine Bedingung und die Alternative – zwei Fälle

Programmzeile	print-Ausgabe
if wert < 10:	
print("wert ist kleiner als 10")	
else:	
print("wert ist größer oder gleich 10")	

Mehrere Bedingungen und die Alternative – vier Fälle

Programmzeile	print-Ausgabe
if wert == 10:	
print("wert ist gleich 10")	
elif wert == 4:	
print("wert ist gleich 4")	
elif wert == 5:	
print("wert ist gleich 5")	
else:	
print("keine Bedingung ist erfüllt")	

Solange die Bedingung wahr ist, ausführen ...



Der Computer dreht Schleifen!

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Gib die Programmzeilen im Editor-Fenster ein.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "schleifen" im Ordner Python/07_Fallunterscheidungen_und_Schleifen
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
Notiere die print-Ausgaben in der Tabelle.

while Schleife

Programmzeile	print-Ausgabe
# Schleifen	
# while-Schleife	
# Variable initialisieren	
durchgang = 1	
while durchgang < 11:	
print(durchgang)	
durchgang = durchgang + 1	
print("nach der Schleife")	

Eine unendliche while Schleife abbrechen – break

Programmzeile	print-Ausgabe
# Variable initialisieren	
durchgang = 1	
# unendliche Schleife mit Abbruch	
while True:	
print("durchgang =", durchgang)	
durchgang = durchgang + 1	
if durchgang > 10:	
break	
print("Nach der Schleife")	

for Schleife – mit Liste

Programmzeile	print-Ausgabe
# Liste anlegen	
vornamen = ["Axel", "Elke", "Martin"]	
# Solange es ein Element in der Liste gibt	
for element in vornamen:	
print(element)	
print("nach der Schleife")	

for Schleife – mit range(stop)

Programmzeile	print-Ausgabe
# Solange es ein Element in der Liste gibt	
for element in range(10):	
print(element)	
print("nach der Schleife")	

Der Computer errät die Zahl



Denke dir eine Zahl und lass den Computer raten!

- Mit dem Menüpunkt **File/ Open ...** öffnest du die Datei " Computer_erraet_die_Zahl.py" im Ordner Python/07_Fallunterscheidungen_und_Schleifen
- In " Computer_erraet_die_Zahl.py" findest du Kommentare mit und ohne Programmzeilen.
- Bearbeite das Programm und **entwickle** die fehlenden Programmzeilen
 - a) Solange, bis der Benutzer "e" eingibt:
Frage den Benutzer "Ist deine Zahl g(rößer), k(leiner), e(rraten)? "
 - b) Bewerte die Benutzereingabe: Wenn der Benutzer "k" eingibt, soll die obere Grenze verkleinert werden, aber nicht kleiner als die untere Grenze.
Tipp: Die Funktion `max(arg1, arg2)` gibt das größte Argument zurück.
 - c) Bewerte die Benutzereingabe: Wenn der Benutzer "g" eingibt, soll die untere Grenze vergrößert werden, aber nicht größer als die obere Grenze.
Tipp: Die Funktion `min(arg1, arg2)` gibt das kleinste Argument zurück.
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- So könnte es aussehen:

```
Denke dir eine Zahl zwischen 1 und 100
Drücke ENTER, wenn du fertig bist
Der Computer rät 50
Ist deine Zahl g(rößer), k(leiner), e(rraten)? g
Der Computer rät 75
Ist deine Zahl g(rößer), k(leiner), e(rraten)? g
Der Computer rät 88
Ist deine Zahl g(rößer), k(leiner), e(rraten)? k
Der Computer rät 81
Ist deine Zahl g(rößer), k(leiner), e(rraten)? k
Der Computer rät 78
Ist deine Zahl g(rößer), k(leiner), e(rraten)? k
Der Computer rät 76
Ist deine Zahl g(rößer), k(leiner), e(rraten)? e
Zahl erraten! Versuche = 6
Fertig?
```

- Durch die Programmzeile `x = (untere + obere)//2` rät der Computer die **Mitte** zwischen zwei Zahlen. Nach maximal 7 Versuchen hat der Computer die Zahl zwischen 1 und 100 erraten!
- Wie viele Versuche benötigt der Computer, um die Zahl zwischen 1 und 1000 zu erraten?

Funktionen in Python

Nach den einfachen Programmen bisher werden wir in den folgenden Kapiteln interessante **Funktionen** schreiben und eine Konstruktionsanleitung einführen.

Warum verwenden wir Funktionen?

Es gibt einige übergreifende Prinzipien des Programmierens, die wir immer wieder aufgreifen werden – **Mantras**. Diese raten uns, **Funktionen** zu verwenden.

Mantra 1

Achte darauf, dass für alles, was Du weißt, tatsächlich auch etwas im Programm steht.

Mantra 2

Lies, was dort steht. Lies die Aufgabenstellung, und zwar Schritt für Schritt, anstatt alles auf einmal. Außerdem gibt IDLE (oder eine andere Programmierumgebung) oft nützliche Rückmeldungen, die es sich zu lesen lohnt.

Mantra 3

Gehe nach der Konstruktionsanleitung vor (folgt).

Mantra 4

Wenn Du zwei Programmteile siehst, die sich nur an wenigen Stellen unterscheiden und die inhaltlich verwandt sind, abstrahiere – schreibe eine **Funktion**, die beides erledigt.

Mantra 5

Wenn in deinem Programm mehrere Dinge berechnet werden, denen du verschiedene Namen geben kannst, schreibe für jedes eine separate **Funktion**.

Mantra 6

Nutze Gleichungen aus, um dein Programm zu vereinfachen.

Mantra 7

Wenn deine **Funktion** dir kompliziert scheint, ist es wahrscheinlich, dass sie Fehler enthält: Entweder, weil sie eigentlich einfacher sein sollte, oder, weil die Aufgabe kompliziert und damit ihre Lösung fehleranfällig ist.

Mantra 8

Suche nach Kombinatoren für deine Daten, die aus kleinen Dingen größere Dinge machen – und daraus wieder größere Dinge machen.

Funktionen haben Input und Output: Ganzzahlen



Funktionen mit Ganzzahlen (Integer) als Input und Output

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Gib die Programmzeilen im Editor-Fenster ein.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "funktionen_io" im Ordner Python/09_Funktionen_Input_und_Output
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
Notiere die print-Ausgaben in der Tabelle.

Funktion ohne Input

Programmzeile	print-Ausgabe
# Funktionen können Input und Output haben	
# Funktion ohne Input	
def ausgabe():	
print("hier bin ich")	
# Aufruf der Funktion	
ausgabe()	

Funktion mit 2 Inputs

Programmzeile	print-Ausgabe
# Funktion mit 2 Inputs	
def ausgabe2a(wert1: int, wert2: int):	
print("wert1 =", wert1, "wert2 =", wert2)	
# Aufruf der Funktion	
ausgabe2a(5, 6)	

Funktion mit 2 Inputs und Vorgabe

Die Werte mit Vorgabe stehen rechts von den Werten ohne Vorgabe.

Programmzeile	print-Ausgabe
# Funktion mit 2 Inputs und Vorgabe	
def ausgabe2b(wert1: int, wert2: int = 15):	
print("wert1 =", wert1, "wert2 =", wert2)	
# Aufruf der Funktion	
ausgabe2b(5)	

Funktion mit 1 Input und 1 Output

Programmzeile	print-Ausgabe
# Funktion mit 1 Input	
def verdoppeln(wert: int) -> int:	
return wert * 2	
# Aufruf der Funktion	
ergebnis = verdoppeln(5)	
print("ergebnis =", ergebnis)	

Funktionen haben Input und Output: Tupel, Liste



Programmzeile	print-Ausgabe
# Funktion mit Tupel als Output	
def wo_bin_ich() -> tuple[int, int]:	
x = 2	
y = 4	
return x, y	
# Aufruf der Funktion	
x, y = wo_bin_ich()	
print("x =", x, "y =", y)	



Achtung: Eine **Liste** ist am Ort veränderbar (mutable object).

Input der Funktion ist eine **Kopie der Liste**, damit das Original unverändert bleibt.

Programmzeile	print-Ausgabe
# Funktion mit einer Liste als Input	
def verteuerung(liste: list[float], p:[float]):	
for i in range(len(liste)):	
liste[i] *= 1 + p	
# Liste anlegen	
original = [9, 12, 12.5, 24.5]	
# Liste kopieren	
kopie = original.copy()	
# Prozentsatz festlegen	
p = 0.05	
# Aufruf der Funktion	
verteuerung(kopie, p)	
# Original und Verteuerung	
print("original =", original)	
print("kopie =", kopie)	

Globale und lokale Variable in der Funktion

Ein Python Modul (eine Datei) hat einen eigenen privaten Namensraum.

Alle Funktionen, die in diesem Modul definiert sind, nutzen ihn als **globalen** Namensraum.



Alle Variablen, die **in** der Funktion nur **gelesen** werden, sind implizit **global**. Beispiel:

```
bspfunktion_global_lesen()
```

Wenn eine Variable **in** der Funktion **einen Wert** bekommt, ist sie **lokal** ...

Beispiel: `bspfunktion_lokal_schreiben()`

... wenn sie nicht **explizit als global** vereinbart wurde.

Beispiel: `bspfunktion_global_lesen_schreiben()`

- Mit dem Menüpunkt **File/ Open ...** öffnest du die Datei "globale_und_lokale_variable_in_der_funktion.py" im Ordner Python/ 09_Funktionen_Input_und_Output.
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
- Lege das Editor-Fenster und das Shell-Fenster nebeneinander auf deinem Bildschirm und vergleiche Programmcode und Ausgabe.

```
1 # globale und lokale Variable in der Funktion
2
3 # Funktion liest eine globale Variable
4 def bspfunktion_global_lesen():
5     print("---- Funktion liest eine globale Variable ----")
6     # die gelesene Variable ist implizit global!
7     print("Variablenwert in Funktion:", variablenWert)
8
9 # Funktion gibt der Variable einen Wert
10 def bspfunktion_lokal_schreiben():
11     print("---- Funktion gibt der Variable einen Wert ----")
12     # die Variable bekommt einen Wert, damit ist sie lokal!
13     variablenWert = "IN der Funktion"
14     print("Variablenwert in Funktion:", variablenWert)
15
16 # Funktion liest eine globale Variable und gibt ihr einen Wert
17 def bspfunktion_global_lesen_schreiben():
18     # die Variable wird als global vereinbart
19     global variablenWert
20     print("---- Funktion liest eine globale Variable und gibt ihr einen Wert ----")
21     # die gelesene globale Variable
22     print("Variablenwert in Funktion 1:", variablenWert)
23     # die globale Variable bekommt einen Wert
24     variablenWert = "IN der Funktion"
25     print("Variablenwert in Funktion 2:", variablenWert)
26
27 # die Variable außerhalb der Funktion ist global
28 variablenWert = "außerhalb der Funktion"
29
30 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
31 # Aufruf der Funktion
32 bspfunktion_global_lesen()
33 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
34
35 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
36 # Aufruf der Funktion
37 bspfunktion_lokal_schreiben()
38 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
39
40 print("\nVariablenwert vor dem Aufruf der Funktion:", variablenWert)
41 # Aufruf der Funktion
42 bspfunktion_global_lesen_schreiben()
43 print("Variablenwert nach dem Aufruf der Funktion:", variablenWert)
```

Funktion konstruieren: Bau den Satz



Bau den Satz!

Gegeben sind drei Listen:

```
subjekt = ["Der Hund", "Die Journalistin", "Der Maler"]
prädikat = ["vergräbt", "interviewt", "malt"]
objekt = ["den Knochen", "den Bürgermeister", "ein Bild"]
```

Schreibe ein Programm, das ein zufälliges Subjekt und ein zufälliges Prädikat und ein zufälliges Objekt hintereinanderstellt und den zufälligen Satz ausgibt.

Du löst die Aufgabe und lernst dabei die **Konstruktionsanleitung**.

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

- Die erste Zeile im Editor-Fenster beschreibt, was das Programm tut:

```
1 | # Das Programm soll zufällige Sätze ausgeben
```

- Du benötigst die Bibliothek für den Zufall, also:

```
3 | # Bibliothek importieren
4 | import random
```

- Die Listen übernimmst du aus der Aufgabenstellung:

```
6 | # Listen
7 | subjekt = ["Der Hund", "Die Journalistin", "Der Maler"]
8 | prädikat = ["vergräbt", "interviewt", "malt"]
9 | objekt = ["den Knochen", "den Bürgermeister", "ein Bild"]
```

- Du schreibst eine Funktion, die den Satz baut. Beginne mit **Punkt 1** der Konstruktionsanleitung:
Kurzbeschreibung.

```
11 | # Kurzbeschreibung
12 | # Die Funktion soll Subjekt, Prädikat, Objekt aus Listen zufällig auswählen
13 | # und einen Satz bauen
```

- **Punkt 2** der Konstruktionsanleitung ist die **Datenanalyse**. Was ist der Input der Funktion, was ist der Output der Funktion:

```
15 | # Datenanalyse
16 | # Input der Funktion sind die (globalen) Listen Subjekt, Prädikat und Objekt
17 | # Output der Funktion ist der Satz
```

- **Punkt 3** der Konstruktionsanleitung ist: **Funktion definieren** **Name** – **Input: Datentyp** – **Output: Datentyp**
Hier geben wir Input: Datentyp nicht an, weil wir mit globalen Listen arbeiten.

```
19 | # Funktion definieren
20 | def bau_den_satz() -> str:
```

- **Punkt 4** der Konstruktionsanleitung ist der **Funktionsrumpf**:

```
21 |     # Funktionsrumpf
22 |     # zufälliges Element der Liste wählen
23 |     mein_subjekt = random.choice(subjekt)
24 |     mein_prädikat = random.choice(prädikat)
25 |     mein_objekt = random.choice(objekt)
26 |     # Satz bauen
27 |     mein_satz = mein_subjekt + " " + mein_prädikat + " " + mein_objekt
28 |     return mein_satz
```

- **Punkt 5** der Konstruktionsanleitung ist: **Ergebnisse prüfen**. Dazu rufen wir die Funktion 5 mal auf.

```
30 | # Ergebnisse prüfen
31 | for i in range(5):
32 |     # Satz bauen
33 |     mein_satz = bau_den_satz()
34 |     # Ergebnis drucken
35 |     print(mein_satz)
```

- **Punkt 6** der Konstruktionsanleitung ist: **Unittest**. Damit würden wir die Funktion mit Input-Daten und erwarteten Output-Daten testen. Diesen Punkt lassen wir hier weg.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "saetze_bauen" im Ordner Python/ 10_Funktionen_konstruieren
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
Notiere die print-Ausgaben

- Vergleiche deine print-Ausgaben mit den print-Ausgaben Anderer.

Funktion konstruieren: GBP in EUR



GBP in EUR umrechnen!

Du möchtest in Großbritannien einkaufen. Die Preise sind dort in britischen Pfund (GBP) angegeben. Du musst also umrechnen.

Schreibe ein Programm, das eine Umrechnungstabelle druckt. In der Tabelle stehen links die Preise in GBP und rechts die zugehörigen Preise in EUR. Die Tabelle geht von 0 GBP bis 10 GBP in Schritten von 0.50 GBP.

Der Umrechnungskurs ist 1 GBP = 1,17 EUR.

Du löst die Aufgabe mit der Konstruktionsanleitung.

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

- Die erste Zeile im Editor-Fenster beschreibt, was das Programm tut:

```
1 | # Das Programm soll GBP in EUR umrechnen und eine Tabelle ausgeben
```

- Du schreibst eine Funktion, die GBP in EUR umrechnet. Beginne mit **Punkt 1** der Konstruktionsanleitung:
Kurzbeschreibung.

```
3 | # Kurzbeschreibung
```

```
4 | # Die Funktion rechnet GBP in EUR um
```

- **Punkt 2** der Konstruktionsanleitung ist die **Datenanalyse**. Was ist der Input der Funktion, was ist der Output der Funktion:

```
6 | # Datenanalyse
```

```
7 | # Input der Funktion ist eine Dezimalzahl in der Einheit GBP
```

```
8 | # Output der Funktion ist eine Dezimalzahl in der Einheit EUR
```

```
9 | # Umrechnungsfaktor 1 GBP = 1.17 EUR
```

- **Punkt 3** der Konstruktionsanleitung ist: **Funktion definieren** Name – Input: Datentyp – Output: Datentyp

```
11 | # Funktion definieren
```

```
12 | def gbp_in_eur(gbp: float) -> float:
```

- **Punkt 4** der Konstruktionsanleitung ist der **Funktionsrumpf**:

```
13 |     eur = gbp * 1.17
```

```
14 |     return eur
```

- **Punkt 5** der Konstruktionsanleitung ist: **Ergebnisse prüfen**.

Dazu legst du die Input-Liste `gbp_liste` an und füllst sie mit Zahlen von 0 bis 10 in Schritten von 0.5.

Und du legst eine leere Output-Liste `eur_liste` an.

```
16 | # Ergebnisse prüfen
```

```
17 | # Input Liste anlegen und füllen
```

```
18 | gbp_liste = []
```

```
19 | for i in range(21):
```

```
20 |     gbp_liste.append(i * 0.5)
```

```
21 | # Output Liste (leer) anlegen
```

```
22 | eur_liste = []
```

Danach rufst du die Funktion für jedes Element in `gbp_liste` auf und hängst den Output der Funktion an `eur_liste` an.

```
24 | # Funktion aufrufen
```

```
25 | for x in gbp_liste:
```

```
26 |     eur_liste.append(gbp_in_eur(x))
```

Dann musst du noch die Tabelle drucken. Zuerst druckst du die Überschrift. Danach bildest du mit `zip(gbp_liste, eur_liste)` ein Tupel mit einem Element aus jeder Liste und druckst es.

```
28 # Ergebnisse drucken
29 print("gbp    eur")
30 for x, y in zip(gbp_liste, eur_liste):
31     print(x, y)
```

- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "gbp_in_eur" im Ordner Python/ 10_Funktionen_konstruieren
- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben



Die Tabelle ist unschön. Die Preise stehen nicht unter den Überschriften. Die Dezimalzahlen haben mehr als 2 Stellen nach dem Komma.

Mit **Format-Strings** kannst du eine schöne Tabelle drucken:

```
28 # Ergebnisse drucken
29 print("gbp    eur")
30 for x, y in zip(gbp_liste, eur_liste):
31     print("{:5.2f} {:5.2f}".format(x, y))
```

- Starte das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben.
Die Format-Strings `{:5.2f}` sorgen dafür, dass die Eingabewerte in einem Feld der Breite 5 mit 2 Stellen nach dem Komma als Fließkommazahl ausgegeben werden.
- **Punkt 6** der Konstruktionsanleitung ist: **Unittest**. Damit testest du die Funktion mit Input-Daten und erwarteten Output-Daten. Was erwartest du, wenn du die Funktion `gbp_in_eur` mit den Input-Daten 0, 1, 6.5, 7.5, 9.5, 10, 50, 100 oder -1 aufrufst?

Wenn du einmal einen Unittest geschrieben hast, muss du die Tabelle nicht selbst checken, sondern kannst das den Computer machen lassen. Du benötigst dazu die Bibliothek `unittest`:

```
33 # Bibliothek importieren
34 import unittest
```

Dann definierst du die Klasse `My_unittest` mit der Testfunktion `test_gbp_in_eur`.
(Wir behandeln Klassen in einer späteren Kurseinheit.)

In der Testfunktion steht der Aufruf der Funktion mit dem Input, dem erwarteten Output und 2.

Die 2 bedeutet, dass beim Vergleich auf 2 Stellen nach dem Komma gerundet wird.

```
36 # Testfunktion definieren
37 class My_unittest(unittest.TestCase):
38
39     def test_gbp_in_eur(self):
40         self.assertEqual(gbp_in_eur(0), 0, 2)
41         self.assertEqual(gbp_in_eur(1), 1.17, 2)
42         self.assertEqual(gbp_in_eur(6.5), 7.6, 2)
43         self.assertEqual(gbp_in_eur(7.5), 8.77, 2)
44         self.assertEqual(gbp_in_eur(9.5), 11.11, 2)
45         self.assertEqual(gbp_in_eur(10), 11.7, 2)
46         self.assertEqual(gbp_in_eur(50), 58.5, 2)
47         self.assertEqual(gbp_in_eur(100), 117, 2)
48         self.assertEqual(gbp_in_eur(-1), -1.17, 2)
```

Nun musst du den `unittest` noch aufrufen:

```
50 # Unittest ausführen
51 if __name__ == '__main__':
52     unittest.main()
```

- **Starte** das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben.

Funktion konstruieren: Kosten für Tennisunterricht



Kosten für Tennisunterricht in Tabelle und Diagramm zeigen

Leistung	Tennis Pro	Go-Tennis
Trainer	30,00 EUR/Stunde	25,00 EUR/Stunde
Platzmiete	20,00 EUR / Stunde	260,00 EUR / Saison

Hinweis: Bei Tennis Pro wird der Platz stundenweise abgerechnet, bei Go-Tennis über eine Saisonpauschale.

Du möchtest Tennisunterricht nehmen. Dazu prüfst du zwei Angebote. Du rechnest mit mindestens 10 Trainerstunden. Welches Angebot ist günstiger?

Schreibe ein Programm, das eine Vergleichstabelle druckt. In der Tabelle stehen links die Trainerstunden, rechts daneben die Kosten für das Angebot Tennis Pro, rechts daneben die Kosten für das Angebot Go-Tennis. Die Tabelle geht von 0 bis 12 Trainerstunden in Schritten von 1 Trainerstunde.

Zeige die Kosten auch in einem Diagramm.

Du löst die Aufgabe mit der Konstruktionsanleitung.

- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".

- Die erste Zeile im Editor-Fenster beschreibt, was das Programm tut:

```
1 | # Das Programm soll GPB in EUR umrechnen und eine Tabelle ausgeben
```

- Für die zwei Angebote brauchst du zwei Funktionen. Beginne mit **Punkt 1** der Konstruktionsanleitung von Tennis Pro: **Kurzbeschreibung**.

```
3 | # Kurzbeschreibung
```

```
4 | # Funktion berechnet die Kosten für das Angebot Tennis Pro
```

- **Punkt 2** der Konstruktionsanleitung von Tennis Pro ist die **Datenanalyse**. Was ist der Input der Funktion, was ist der Output der Funktion:

```
6 | # Datenanalyse
```

```
7 | # Input der Funktion ist eine Dezimalzahl in der Einheit Stunde
```

```
8 | # Output der Funktion ist eine Dezimalzahl in der Einheit EUR
```

```
9 | # Trainer: 30 EUR/Stunde Platzmiete: 20 EUR/Stunde
```

Hinweis: Die Preise der Angebote sind ganzzahlig. Du könntest Input und Output als Ganzzahl (`int`) angeben. Gib Input und Output als `float` an, denn die Preise können sich ändern! Die Funktion soll auch mit Dezimalzahlen arbeiten können.

- **Punkt 3** der Konstruktionsanleitung von Tennis Pro ist:

Funktion definieren **Name** – **Input: Datentyp** – **Output: Datentyp**

```
11 | # Funktion definieren
```

```
12 | def tennis_pro(trainerstunden: float) -> float:
```

- **Punkt 4** der Konstruktionsanleitung von Tennis Pro ist der **Funktionsrumpf**:

```
13 |     kosten = (20 + 30) * trainerstunden
```

```
14 |     return kosten
```

- Es geht weiter mit **Punkt 1** der Konstruktionsanleitung von Go-Tennis: **Kurzbeschreibung**.

```
16 | # Kurzbeschreibung
17 | # Funktion berechnet die Kosten für das Angebot Go-Tennis
```

- **Punkt 2** der Konstruktionsanleitung von Go-Tennis ist die **Datenanalyse**. Was ist der Input der Funktion, was ist der Output der Funktion:

```
19 | # Datenanalyse
20 | # Input der Funktion ist eine Dezimalzahl in der Einheit Stunde
21 | # Output der Funktion ist eine Dezimalzahl in der Einheit EUR
22 | # Trainer: 25 EUR/Stunde   Platzmiete: 260 EUR/Saison
```

- **Punkt 3** der Konstruktionsanleitung von Go-Tennis ist:

Funktion definieren Name – Input: Datentyp – Output: Datentyp

```
24 | # Funktion definieren
25 | def go_tennis(trainerstunden: int) -> float:
```

- **Punkt 4** der Konstruktionsanleitung von Go-Tennis ist der **Funktionsrumpf**:

```
26 |     kosten = 260 + 25 * trainerstunden
27 |     return kosten
```

- **Punkt 5** der Konstruktionsanleitung ist: **Ergebnisse prüfen**.

Dazu legst du die Input-Liste `trainerstunden_liste` an und füllst sie mit Zahlen von 0 bis 12.

Für jedes Angebot legst eine leere Output-Liste an:

`tennis_pro_kosten_liste` und `go_tennis_kosten_liste`

```
29 | # Ergebnisse prüfen
30 | # Input Liste anlegen
31 | trainerstunden_liste = range(0, 13)
32 | # Output Liste (leer) von Tennis Pro anlegen
33 | tennis_pro_kosten_liste = []
34 | # Output Liste (leer) von Go-Tennis anlegen
35 | go_tennis_kosten_liste = []
```

Danach rufst du beide Funktionen für jedes Element in `trainerstunden_liste` auf.

Den Output der Funktion `tennis_pro` hängst du an die Output-Liste `tennis_pro_kosten_liste` an.

Den Output der Funktion `go_tennis` hängst du an die Output-Liste `go_tennis_kosten_liste` an.

```
37 | # Funktionen aufrufen
38 | for x in trainerstunden_liste:
39 |     tennis_pro_kosten_liste.append(tennis_pro(x))
40 |     go_tennis_kosten_liste.append(go_tennis(x))
```

Dann musst du noch die Tabelle drucken. Zuerst druckst du die Überschrift. Danach bildest du mit

`zip(trainerstunden_liste, tennis_pro_kosten_liste, go_tennis_kosten_liste)`

ein Tupel mit einem Element aus jeder Liste und druckst es. Mit **Format-Strings** wird es eine schöne Tabelle.

```
42 | # Ergebnisse drucken
43 | print("Trainerstunden   Tennis Pro   Go-Tennis")
44 | for x, y, z in zip(trainerstunden_liste, tennis_pro_kosten_liste, go_tennis_kosten_liste):
45 |     print("{:14.2f}   {:10.2f}   {:9.2f}".format(x, y, z))
```

- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "tennisunterricht_kosten" im Ordner Python/ 10_Funktionen_konstruieren
- **Starte** das Programm mit dem Menüpunkt **Run/ Run Module**. Betrachte die print-Ausgaben.
- **Punkt 6** der Konstruktionsanleitung ist: **Unittest**. Damit würden wir die Funktion mit Input-Daten und erwarteten Output-Daten testen. Diesen Punkt lassen wir hier weg.

- Die Kosten sollen in einem Diagramm gezeigt werden. Du benötigst dazu die Bibliothek `matplotlib.pyplot`. Der Name wird mit `plt` abgekürzt.

```
47 | # Bibliothek importieren
48 | import matplotlib.pyplot as plt
```

Da die Ergebnisse in **Listen** vorliegen, kannst du sie mit wenigen Befehlen plotten:

```
50 | # Ergebnisse plotten
51 | plt.plot(trainerstunden_liste, tennis_pro_kosten_liste)
52 | plt.plot(trainerstunden_liste, go_tennis_kosten_liste)
53 | plt.xlabel("Trainerstunden")
54 | plt.ylabel("Kosten")
55 | plt.show()
```

- **Starte** das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben und die Grafik-Ausgabe.

Klassen haben Eigenschaften und Methoden

Eine Instanz der Klasse beschreibt ein konkretes Objekt mit Eigenschaften und Methoden

BauplanKatzenKlasse bauen und zwei Instanzen erstellen – Katzen_Klasse.py

```

1  # Objektorientierte Programmierung
2
3  # Klassendefinition
4  class BauplanKatzenKlasse():
5      """ Klasse für das Erstellen von Katzen
6      Hilfetext ideal bei mehreren Programmierern in
7      einem Projekt oder bei schlechtem Gedächtnis """
8
9      # Methoden der Klasse
10     def __init__(self, rufname, farbe, alter):
11         self.rufname = rufname
12         self.farbe = farbe
13         self.alter = alter
14         # Dauer aufaddieren
15         self.schlafdauer = 0
16
17     def tut_miauen(self, anzahl = 1):
18         print(anzahl * "miau ")
19
20     def tut_schlafen(self, dauer):
21         print(self.rufname, "schläft jetzt", dauer , "Minuten ")
22         self.schlafdauer += dauer
23         print(self.rufname, "Schlafdauer insgesamt:", self.schlafdauer, "Minuten ")
24
25 # Instanz Sammy erstellen
26 katze_sammy = BauplanKatzenKlasse("Sammy", "orange", 3)
27 print(katze_sammy.rufname)
28 print(katze_sammy.farbe)
29 print(katze_sammy.alter)
30
31 # Instanz Soni erstellen
32 katze_soni = BauplanKatzenKlasse("Soni", "getigert", 2)
33
34 # Methoden von Katze Sammy aufrufen
35 katze_sammy.tut_miauen(3)
36 katze_sammy.tut_schlafen(3)
37 katze_sammy.tut_schlafen(6)
38 katze_sammy.tut_miauen(5)
39 katze_sammy.tut_schlafen(10)
40
41 # Methode von Katze Soni aufrufen
42 katze_soni.tut_schlafen(5)

```

Der Klassenname beginnt mit einem Großbuchstaben

In der Methode `__init__` stehen die Eigenschaften
In der Klammer kommt immer zuerst "self"

Methoden:
Was eine Katze kann!

Wenn wir die Instanz "katze_sammy" erstellen, erhält "self" den Namen "katze_sammy". Die Eigenschaften erhalten die Werte in der Klammer.

Wenn wir die Methode "tut_miauen" aufrufen, erhält "self" den Namen "katze_sammy". "anzahl" erhält den Wert in der Klammer.

Baue deine eigene Klasse



Baue die Klasse Fahrrad und erstelle zwei Instanzen!

Programmiere die Klasse "Fahrrad" mit den Eigenschaften: Besitzer, Farbe, Typ, Anzahl Gänge

"Typ" ist: Touring, Renn, Mountain

Programmiere die Methoden: klingeln, fahren

Erstelle zwei Instanzen der Klasse "BauplanFahrradKlasse" und gib die Eigenschaften aus.

Rufe die Methoden der Instanzen auf.

Das Programm Katzen_Klasse.py löst eine ganz ähnliche Aufgabe. Nimm Katzen_Klasse.py als **Vorbild** und schreibe das Programm Fahrrad_Klasse.py.

- Mit dem Menüpunkt **File/ Open ...** öffnest du die Datei "Katzen_Klasse.py" im Ordner Python/11_OOP_Klassen.
- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Lege das Fenster mit dem Titel "Katzen_Klasse.py" in die linke Hälfte deines Bildschirms.
Lege das Fenster mit dem Titel "untitled" in die rechte Hälfte deines Bildschirms.
Im Folgenden schreibst du ausschließlich in das Fenster "untitled".
- Die erste Zeile im Editor-Fenster beschreibt, was das Programm tut:
1 | `# Objektorientierte Programmierung`
- Die Klassendefinition und den Hilfetext machst du nach dem **Vorbild**:
Anstelle von `BauplanKatzenKlasse` schreibst du `BauplanFahrradKlasse`
Anstelle von `Katzen` schreibst du `Fahrrädern`
- Die Methode `__init__` der Klasse definierst du nach dem **Vorbild**:
Anstelle von `rufname, farbe, alter` schreibst du `besitzer, farbe, typ, gaenge`
Anstelle von `self.rufname = rufname` schreibst du `self.besitzer = besitzer`
usw.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "Fahrrad_Klasse" im Ordner Python/11_OOP_Klassen.
- Ab hier kannst du die Klassendefinition alleine vervollständigen!
Beantworte die Fragen und programmiere nach dem **Vorbild**.
Welche Methode wird nach dem Vorbild `tut_miauen` programmiert?
Welche Methode wird nach dem Vorbild `tut_schlafen` programmiert?
Welche Größe muss in der Methode `__init__` aufaddiert werden?
- Erstelle die Instanzen der Klasse "BauplanFahrradKlasse" nach dem **Vorbild**.
- Rufe die Methoden der Instanzen nach dem **Vorbild** auf.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "Fahrrad_Klasse" im Ordner Python/11_OOP_Klassen.
- **Starte** das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben.

Die Eltern-Klasse vererbt – die Kind-Klasse erbt

Die Kind-Klasse erbt alle Eigenschaften und Methoden der Eltern-Klasse.

Eine Eltern-Klasse und zwei Kind-Klassen programmieren – Tier_Klasse.py

```

1 # Vererbung in Python
2
3 # Eltern-Klasse
4 class Tier():
5     """ Klasse für das Erstellen von Säugetieren """
6
7     def __init__(self, rufname, farbe, alter):
8         self.rufname = rufname
9         self.farbe = farbe
10        self.alter = alter
11        self.schlafdauer = 0
12
13    def tut_reden(self, anzahl = 1):
14        print(self.rufname, "sagt: ", anzahl * "miau ")
15
16    def tut_schlafen(self, dauer):
17        print(self.rufname, "schläft jetzt", dauer , "Minuten ")
18        self.schlafdauer += dauer
19        print(self.rufname, "Schlafdauer insgesamt:", self.schlafdauer, "Minuten")
20
21 # Kind-Klasse für Katzen
22 class BauplanKatzenKlasse(Tier):
23     """ Klasse für das Erstellen von Katzen """
24
25     def __init__(self, rufname, farbe, alter):
26         """ Initialisieren über Eltern-Klasse """
27         super().__init__(rufname, farbe, alter)
28
29 # Kind-Klasse für Hunde
30 class Hund(Tier):
31     """ Klasse für das Erstellen von Hunden """
32
33     def __init__(self, rufname, farbe, alter):
34         """ Initialisieren über Eltern-Klasse """
35         super().__init__(rufname, farbe, alter)
36
37     def tut_reden(self, anzahl = 1):
38         """ Überschreiben der Methode der Eltern-Klasse """
39         print(self.rufname, "sagt: ", anzahl * "WAU ")
40
41 # Instanz Sammy erstellen
42 katze_sammy = BauplanKatzenKlasse("Sammy", "orange", 3)
43 print(katze_sammy.farbe)
44
45 # Instanz Bello erstellen
46 hund_bello = Hund("Bello", "braun", 5)
47 print(hund_bello.farbe)
48
49 # Methoden aufrufen
50 hund_bello.tut_schlafen(4)
51 katze_sammy.tut_schlafen(5)
52 hund_bello.tut_schlafen(2)
53 katze_sammy.tut_reden(1)
54 hund_bello.tut_reden(3)

```

"Tier" ist der Oberbegriff für Katzen und Hunde

Die Eltern-Klasse Tier bekommt Eigenschaften, damit etwas vererbt werden kann.

Methoden:
Was ein Tier kann!

In der Klammer steht die Eltern-Klasse von BauplanKatzenKlasse

Die Methode __init__ wird weiterhin benötigt. super() holt die Eigenschaften der Eltern-Klasse ab. Die Methoden werden automatisch geerbt.

In der Klammer steht die Eltern-Klasse von Hund

Die Methode __init__ wird weiterhin benötigt. super() holt die Eigenschaften der Eltern-Klasse ab. Die Methoden werden automatisch geerbt.

Die Methode mit dem exakt gleichen Namen überschreibt die Methode der Eltern-Klasse.

Instanzen "katze_sammy" und "hund_bello" erstellen

Wenn wir die Methode hund_bello.tut_reden() aufrufen, bellt der Hund!

Baue Eltern-Klasse und Kind-Klassen



Baue die Eltern-Klasse Zweirad und die Kind-Klassen Fahrrad und Pedelec

Programmiere die Eltern-Klasse "Zweirad" mit den Eigenschaften: Besitzer, Farbe, Typ, Anzahl Gänge und den Methoden: fahren, klingeln.

Erstelle die Kind-Klassen "Fahrrad" und "Pedelec", die alle Eigenschaften und Methoden der Eltern-Klasse "Zweirad" erben.

Die Klasse "Pedelec" hat zusätzlich die Eigenschaft "kapazitaet" (Wattstunden) und die Methode "aufladen".

Erstelle eine Instanz der Klasse "Fahrrad" und eine Instanz der Klasse "Pedelec"

Gib die Eigenschaft "besitzer" der Instanz des "Fahrrad" aus.

Gib die Eigenschaften "besitzer" und "kapazitaet" der Instanz des "Pedelec" aus.

Rufe alle Methoden der Instanzen auf.

Das Programm Tier_Klasse.py löst eine ganz ähnliche Aufgabe. Nimm Tier_Klasse.py als **Vorbild** und schreibe das Programm Zweirad_Klasse.py.

- Mit dem Menüpunkt **File/ Open ...** öffnest du die Datei "Tier_Klasse.py" im Ordner Python/12_OOP_Verbung_bei_Klassen.
- Mit dem Menüpunkt **File/ New File** erzeugst du ein leeres Editor-Fenster mit dem Titel "untitled".
- Lege das Fenster mit dem Titel "Tier_Klasse.py" in die linke Hälfte deines Bildschirms. Lege das Fenster mit dem Titel "untitled" in die rechte Hälfte deines Bildschirms. Im Folgenden schreibst du ausschließlich in das Fenster "untitled".
- Die erste Zeile im Editor-Fenster beschreibt, was das Programm tut:

```
1 | # Vererbung in Python
```

- Die Definition Eltern-Klasse und den Hilfetext machst du nach dem **Vorbild**:
Anstelle von `Tier` schreibst du `Zweirad`
Anstelle von `Säugetieren` schreibst du `Zweirädern`
- Die Methode `__init__` der Klasse definierst du nach dem **Vorbild**:
Anstelle von `rufname, farbe, alter` schreibst du `besitzer, farbe, typ, gaenge`
Anstelle von `self.rufname = rufname` schreibst du `self.besitzer = besitzer`
usw.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "Zweirad_Klasse" im Ordner Python/12_OOP_Verbung_bei_Klassen.
- Ab hier kannst du die Definition der Eltern-Klasse alleine vervollständigen!
Beantworte die Fragen und programmiere nach dem **Vorbild**.
Welche Methode wird nach dem Vorbild `tut_miauen` programmiert?
Welche Methode wird nach dem Vorbild `tut_schlafen` programmiert?
Welche Größe muss in der Methode `__init__` aufaddiert werden?
- Die Definition der Kind-Klasse für Fahrräder und den Hilfetext machst du nach dem **Vorbild**:
Anstelle von `Katzen` schreibst du `Fahrräder`
Anstelle von `BauplanKatzenKlasse` schreibst du `BauplanFahrradKlasse`
Anstelle von `Katzen` schreibst du `Fahrrädern`

- Die Methode `__init__` der Kind-Klasse für Fahrräder machst du nach dem **Vorbild**:
Anstelle von `rufname, farbe, alter` schreibst du `besitzer, farbe, typ, gaenge`
Das musst du an zwei Stellen machen!
- Die Definition der Kind-Klasse für Pedelecs und den Hilfetext machst du nach dem **Vorbild**:
Anstelle von `Hunde` schreibst du `Pedelecs`
Anstelle von `Hund` schreibst du `Pedelec`
Anstelle von `Hunden` schreibst du `Pedelecs`
- Die Methode `__init__` der Kind-Klasse für Pedelecs machst du nach dem **Vorbild**:
Anstelle von `rufname, farbe, alter` schreibst du `besitzer, farbe, typ, gaenge`
Das musst du an zwei Stellen machen!
Die Eigenschaft `kapazitaet` schreibst zusätzlich in die Klammer der Methode `__init__`.
Die Eigenschaft `kapazitaet` wird nicht von der Eltern-Klasse abgeholt, weil sie dort nicht vorhanden ist.
- Die Methode `aufladen` der Kind-Klasse für Pedelecs machst du nach dem **Vorbild**:
Anstelle von `tut_reden(self, anzahl = 1)` schreibst du `aufladen(self)` mit dem Rumpf:

```
print("Der Akku ist ausreichend geladen")
```
- Erstelle die Instanzen der Kind-Klassen nach dem **Vorbild**.
- Gib die Eigenschaften "besitzer" und "kapazitaet" (falls vorhanden) der Instanzen aus.
- Rufe alle Methoden der Instanzen auf.
- Speichere den Inhalt mit dem Menüpunkt **File/ Save** unter dem Dateinamen "Zweirad_Klasse" im Ordner Python/12_OOP_Vererbung_bei_Klassen.
- **Starte** das Programm mit dem Menüpunkt **Run/ Run Module**.
Betrachte die print-Ausgaben.

Roboter im Irrgarten: Wände bauen

Wir arbeiten mit der grafischen Benutzeroberfläche "Turtle"

Einen Irrgarten bauen – Irrgarten_Klasse.py

Das Programm bringt ein Fenster mit 3 Blöcken unterschiedlicher Farbe auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe

Setze Blöcke nebeneinander und untereinander, um Wände zu bauen. Experimentiere mit den Farben.

```

1 # Programm erzeugt ein Fenster und setzt die Wände eines Irrgartens hinein
2 # - Eine Funktion setzt die Wände in das Fenster
3 # Modul für die Turtle-Grafik importieren
4 import turtle
5 # Irrgarten-Klasse
6 class Maze:
7     """Klasse für den Bau eines Irrgartens"""
8     # Methoden der Klasse
9     def __init__(self):
10         self.rows_in_maze = 11
11         self.columns_in_maze = 22
12         # turtle Objekt erzeugen und Aussehen festlegen
13         self.t = turtle.Turtle()
14         self.t.shape("turtle")
15         # Breite, Höhe und Koordinaten des Fensters festlegen
16         self.wn = turtle.Screen()
17         self.wn.setup(800, 400)
18         self.wn.setworldcoordinates(0, 0, self.columns_in_maze, self.rows_in_maze)
19     # Zeichne ein ausgefülltes Rechteck
20     def draw_box(self, x, y, color):
21         self.t.up() # Stift hoch
22         self.t.goto(x, y)
23         self.t.color(color)
24         self.t.fillcolor(color)
25         self.t.setheading(90)
26         self.t.down() # Stift runter
27         self.t.begin_fill()
28         # Rechteck zeichnen und füllen
29         for i in range(4):
30             self.t.forward(1)
31             self.t.right(90)
32         self.t.end_fill()
33     # Setze Wände in das Fenster
34     def draw_maze(self):
35         # Farben: 'white', 'black', 'red', 'green', 'blue', 'cyan', 'yellow', 'magenta'
36         self.draw_box(0, 0, "orange")
37         self.draw_box(self.columns_in_maze - 1, self.rows_in_maze - 1, "magenta")
38         self.draw_box(self.columns_in_maze//2, self.rows_in_maze//2, "cyan")
39         # Farbe der Schildkröte
40         self.t.color("black")
41         self.t.fillcolor("blue")
42         self.wn.update()
43 # Instanz erzeugen
44 my_maze = Maze()
45 # Irrgarten zeichnen
46 my_maze.draw_maze()
47 my_maze.t.up() # Stift hoch
48 # Schildkröte in Home-Position
49 my_maze.t.home()
50 # Turtle event loop
51 my_maze.wn.mainloop()

```

Roboter im Irrgarten: Wände ablegen

Wände des Irrgartens in einer Liste ablegen - Irrgarten_Klasse_maze_list.py

Überall wo ein '+' in der Liste steht, zeichnet das Programm einen Block in das Fenster. In der Mitte des Fensters formen die Wände ein Wort. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Ersetze in Zeile 62 <code>self.from_bottom(row)</code> durch <code>row</code>	
Kommentiere die Zeile 58 aus	

[illegible]

Roboter im Irrgarten: Wände aus Datei lesen

Wände des Irrgartens aus einer Datei lesen - Irrgarten_Klasse_maze_filename.py

Das Programm öffnet die Datei `maze3.txt` und kopiert alle Zeilen in der Datei in die Variable `lines`. Dann geht das Programm durch jede Zeile von `lines` und erzeugt eine Liste mit allen Zeichen der Zeile. Diese Liste wird an `maze_list` angehängt. Starte das Programm.

Ändere die Datei `maze3.txt` und starte das Programm erneut.

Aufgabe	Ergebnis
Vergleiche die '+' in <code>maze3.txt</code> mit den Blöcken im Fenster	
Ergänze und entferne '+' in <code>maze3.txt</code> und vergleiche erneut	

Inhalt der Datei `maze3.txt`

```

+++++
+      ++ ++  +  +
+    +++++ ++ +  +++  +
+      +  +  +  +  +
+ ++++++  +  +  +++  +
+      +  +  +  +++  +
+ +++++++  +  +  +
+      +  +  +  +  +
+ +++++++  +  +  +
+      +  +  +  +  +
+      S  +  +
+++++

```

```

10 # Irrgarten-Klasse
11 class Maze:
12     """Klasse für den Bau eines Irrgartens"""
13     # Methoden der Klasse
14     def __init__(self, maze_filename):
15         # maze_list aus Datei lesen
16         self.maze_list = []
17         with open(maze_filename, "r") as maze_file:
18             self.lines = maze_file.readlines()
19             for line in self.lines:
20                 self.maze_list.append([ch for ch in line.rstrip("\n")])
21             self.rows_in_maze = len(self.maze_list)
22             self.columns_in_maze = len(self.maze_list[0])
23             print("rows in maze =", self.rows_in_maze, "columns in maze =", self.columns_in_maze)
24
25 # Instanz erzeugen
26 my_maze = Maze("maze3.txt")
27 # Irrgarten zeichnen
28 my_maze.draw_maze()
29 my_maze.t.up() # Stift hoch
30 # Schildkröte in Home-Position
31 my_maze.t.home()
32 # Turtle event loop
33 my_maze.wn.mainloop()

```

Roboter im Irrgarten: Schildkröte bewegen

Schildkröte durch den Irrgarten bewegen - Irrgarten_Klasse_move_turtle.py

Nachdem das Programm die Liste `maze_list` erstellt hat, sucht es in `maze_list` nach dem Startpunkt "S".

Die Funktion `search_from()` setzt die Schildkröte auf den Startpunkt. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Vergleiche das 'S' in <code>maze3.txt</code> mit der Position der Schildkröte	
Schreibe hinter die Zeile 89: <code>maze.update_position(start_row - 2, start_col)</code>	

```

12 # Irrgarten-Klasse
13 class Maze:
14     """Klasse für den Bau eines Irrgartens"""
15     # Methoden der Klasse
16     def __init__(self, maze_filename):
17         # maze_list aus Datei lesen
18         self.maze_list = []
19         with open(maze_filename, "r") as maze_file:
20             self.lines = maze_file.readlines()
21             for line in self.lines:
22                 self.maze_list.append([ch for ch in line.rstrip("\n")])
23         self.rows_in_maze = len(self.maze_list)
24         self.columns_in_maze = len(self.maze_list[0])
25         print("rows_in_maze =", self.rows_in_maze, "columns_in_maze =", self.columns_in_maze)
26         # Start in maze_list suchen
27         for row in range(self.rows_in_maze):
28             for col in range(self.columns_in_maze):
29                 if self.maze_list[row][col] == START:
30                     self.start_row = row
31                     self.start_col = col
32                     break
33         print("start_row =", self.start_row, "start_col =", self.start_col)
34
35     # Bewege die Schildkröte
36     def move_turtle(self, x, y):
37         x += 0.5
38         y += 0.5
39         self.t.setheading(self.t.towards(x, y))
40         self.t.goto(x, y)
41
42     # Aktualisiere die Position der Schildkröte
43     def update_position(self, row, col, val=None):
44         # Wenn val angegeben: Element der Liste überschreiben
45         if val:
46             self.maze_list[row][col] = val
47             self.move_turtle(col, self.from_bottom(row))
48
49 # Roboter sucht den Weg, Schildkröte läuft mit
50 def search_from(maze, start_row, start_col):
51     # Schildkröte auf die Startposition setzen
52     maze.update_position(start_row, start_col, BLANK)
53     # Der Roboter sucht den Weg ... folgt
54
55 # Instanz erzeugen
56 my_maze = Maze("maze3.txt")
57 # Irrgarten zeichnen
58 my_maze.draw_maze()
59 my_maze.t.up() # Stift hoch
60 # Schildkröte in Home-Position
61 my_maze.t.home()
62 # Suche den Weg vom Start zum Ausgang
63 search_from(my_maze, my_maze.start_row, my_maze.start_col)
64
65 # Turtle event loop
66 my_maze.wn.mainloop()

```

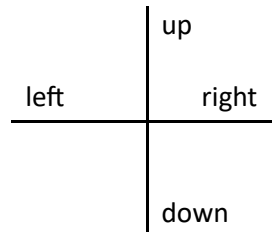
Roboter im Irrgarten: Schauen, gehen, drehen

Roboter schaut, geht und dreht - Irrgarten_Klasse_robot_methods.py

Das Programm kann die Schildkröte bewegen. Die Schildkröte erkennt aber keine Wände. Wir benötigen also einen Roboter, der schaut, geht und dreht. Die Position des Roboters wird durch seine `row` und `col` dargestellt.

Der Roboter soll Folgendes können:

- in alle 4 Richtungen schauen
- in alle 4 Richtungen gehen
- nach rechts drehen
- nach links drehen



Rechts und links sind abhängig von der aktuellen Richtung. Ein Dictionary gibt uns die neue Richtung für rechts und für links. Beispiel: Die aktuelle Richtung ist "down". Rechts ist "left".

In eine Richtung schauen oder gehen bedeutet:

- `row` unverändert oder `row + 1` oder `row - 1` *und*
- `col` unverändert oder `col + 1` oder `col - 1`

Ein Dictionary gibt uns das für `row` und `col` vor. Beispiel: "left" bedeutet `row` unverändert *und* `col - 1`.

Die Klasse `Maze` bekommt zusätzlich 6 Roboter-Methoden. Wir testen die Methoden mit Hilfe der Funktion `search_from()`. Die Funktion enthält eine Benutzerabfrage. Dort gebt ihr ein Kommando ein, und der Roboter schaut, geht und dreht. Starte das Programm. Gib Kommandos ein und beobachte die Reaktion von Roboter und Schildkröte. Tipp: Das Kommando `h[and]` steht für: Schau zur rechten Hand.

Aufgabe	Ergebnis
Gib die Kommandos <code>v[orne]</code> , <code>h[and]</code> , <code>l[inks]</code> , <code>r[echts]</code> , <code>s[chritt]</code> , <code>e[nde]</code> ein und bewege den Roboter eine kurze Strecke durch den Irrgarten. Kann der Roboter durch Wände gehen?	

```

14 # Richtungs-Eigenschaften
15 right_of = {"up": "right", "down": "left", "left": "up", "right": "down"}
16 left_of  = {"up": "left", "down": "right", "left": "down", "right": "up"}
17 angle_of = {"up": 90, "down": 270, "left": 180, "right": 0}
18 delta_row = {"up": -1, "down": 1, "left": 0, "right": 0}
19 delta_col = {"up": 0, "down": 0, "left": -1, "right": 1}

94 # Exit erreicht?
95 def is_exit(self, row, col):
96     return (
97         row == 0
98         or row == self.rows_in_maze - 1
99         or col == 0
100        or col == self.columns_in_maze - 1
101    )
102 # Geschwindigkeit und Richtung der Schildkröte einstellen
103 def init_search(self, heading):
104     self.t.speed(3)
105     self.t.pendown() # Stift runter
106     self.t.setheading(angle_of[heading])
107 # schau nach vorne
108 def look_forward(self, start_row, start_col, heading):
109     # im Exit nicht nach vorne schauen!
110     if self.is_exit(start_row, start_col) == True:
111         return EXIT
112     else:
113         return self.maze_list[start_row + delta_row[heading]]\
114                [start_col + delta_col[heading]]
115 # schau nach rechts
116 def look_right(self, start_row, start_col, heading):
117     return self.maze_list[start_row + delta_row[right_of[heading]]\
118                        [start_col + delta_col[right_of[heading]]]

```



```

119     # mache einen Schritt
120     def one_step(self, start_row, start_col, heading):
121         start_row += delta_row[heading]
122         start_col += delta_col[heading]
123         return start_row, start_col
124     # drehe dich nach rechts
125     def turn_right(self, heading):
126         self.t.right(90)
127         return right_of[heading]
128     # drehe dich nach links
129     def turn_left(self, heading):
130         self.t.left(90)
131         return left_of[heading]
132 # Roboter sucht den Weg, Schildkröte läuft mit
133 def search_from(maze, start_row, start_col):
134     # Schildkröte auf die Startposition setzen
135     maze.update_position(start_row, start_col, BLANK)
136     # Richtung festlegen
137     heading = "up"
138     print(heading)
139     # Schildkröte einstellen
140     maze.init_search(heading)
141     # cmd initialisieren
142     cmd = ""
143     # Solange wiederholen, bis Benutzer "e" eingibt
144     while cmd != "e":
145         # cmd leeren
146         cmd = ""
147         # Solange wiederholen, bis Benutzer etwas eingibt
148         while cmd == "":
149             cmd = input("Kommando v[orne], h[and], l[inks], r[echts], s[chritt], e[nde] ")
150             print("Kommando =", cmd)
151             if maze.is_exit(start_row, start_col) == True:
152                 print("Turtle im Exit!")
153                 print("Ende - du kannst das Turtle-Fenster jetzt schließen")
154                 break
155             elif cmd == "v":
156                 ch = maze.look_forward(start_row, start_col, heading)
157                 print(ch)
158             elif cmd == "h":
159                 ch = maze.look_right(start_row, start_col, heading)
160                 print(ch)
161             elif cmd == "l":
162                 heading = maze.turn_left(heading)
163                 print(heading)
164             elif cmd == "r":
165                 heading = maze.turn_right(heading)
166                 print(heading)
167             elif cmd == "s":
168                 start_row, start_col = maze.one_step(start_row, start_col, heading)
169                 maze.update_position(start_row, start_col)
170                 print("start_row =", start_row, "start_col =", start_col)
171             elif cmd == "e":
172                 print("Ende - du kannst das Turtle-Fenster jetzt schließen")
173             else:
174                 print("Unbekanntes Kommando")
175     # Der Roboter sucht den Weg ... folgt

```


Roboter im Irrgarten: Rechte-Hand-Methode

Mit der Rechte-Hand-Methode findet der Roboter aus dem Irrgarten -
[turtle_in_maze_right_hand_rule_2x1.py](#)

Die Methoden `look_forward()` und `look_right()` melden eine der folgenden Situationen.

vorne frei



vorne Wand



Wand rechts



Wand nicht rechts



Der Pfeil zeigt, wie der Roboter reagieren soll, damit seine rechte Hand an der Wand bleibt.

Der Roboter hat den Ausgang erreicht, wenn die Methode `is_exit()` `True` meldet. Starte das Programm.

Aufgabe	Ergebnis
Ersetze nun in <code>maze3.txt</code> Zeile 8 Spalte 17 das <code>+</code> durch ein Leerzeichen. Findet der Roboter den Weg aus dem Irrgarten? Wie muss der Irrgarten beschaffen sein, damit der Roboter hinausfindet?	

```

131 def search_from(maze, start_row, start_col):
132     # Schildkröte auf die Startposition setzen
133     maze.update_position(start_row, start_col, BLANK)
134     # Richtung festlegen
135     heading = "up"
136     print(heading)
137     # Schildkröte einstellen
138     maze.init_search(heading)
139
140     # solange vorne frei ist
141     while maze.look_forward(start_row, start_col, heading) == BLANK:
142         start_row, start_col = maze.one_step(start_row, start_col, heading)
143         maze.update_position(start_row, start_col)
144         # drehen, damit rechte Hand an der Wand ist
145         heading = maze.turn_left(heading)
146
147     # Drehungen zählen
148     turn_count = 1
149
150     # Folge der Wand bis Exit
151     while maze.is_exit(start_row, start_col) == False:
152
153         # solange vorne frei und die Wand rechts ist
154         while maze.look_forward(start_row, start_col, heading) == BLANK\
155             and maze.look_right(start_row, start_col, heading) == OBSTACLE:
156             start_col = maze.one_step(start_row, start_col, heading)
157             maze.update_position(start_row, start_col)
158
159         # wenn Exit, dann Schleife abbrechen
160         if maze.is_exit(start_row, start_col) == True:
161             break
162
163         # wenn die Wand nicht mehr rechts ist
164         elif maze.look_right(start_row, start_col, heading) == BLANK:
165             # drehen und 1 Schritt vorwärts, damit rechte Hand an der Wand ist
166             heading = maze.turn_right(heading)
167             turn_count += 1
168             start_row, start_col = maze.one_step(start_row, start_col, heading)
169             maze.update_position(start_row, start_col)
170
171         # wenn vorne eine Wand ist
172         elif maze.look_forward(start_row, start_col, heading) == OBSTACLE:
173             # drehen, damit rechte Hand an der Wand ist
174             heading = maze.turn_left(heading)
175             turn_count += 1
176
177     # Ende der while-Schleife
178     print("Exit found!")
179     print(turn_count, "Drehungen")

```

Roboter im Irrgarten: Pledge-Algorithmus

Mit dem Pledge-Algorithmus findet der Roboter aus jedem Irrgarten - `turtle_in_maze_pledge_algorithm_2x1.py`

Der Pledge-Algorithmus arbeitet mit dem Drehungs-Level `turn_level`. Zu Beginn ist der Drehungs-Level Null. Bei einer Linksdrehung wird er um 1 erhöht, bei einer Rechtsdrehung um 1 erniedrigt. Der Roboter läuft geradeaus bis zur Wand und macht eine Linksdrehung, damit die rechte Hand an die Wand ist. Danach folgt er der Wand bis zum Ausgang oder bis der Drehungs-Level Null ist. Wenn der Drehungs-Level Null ist, läuft der Roboter wieder geradeaus bis zur Wand, ohne auf die rechte Hand zu achten. Starte das Programm.

Aufgabe	Ergebnis
Ersetze nun in <code>maze3.txt</code> Zeile 8 Spalte 17 das + durch ein Leerzeichen. Findet der Roboter den Weg aus dem geänderten Irrgarten <code>maze3.txt</code> ?	
Teste das Programm mit dem Irrgarten <code>my_maze.txt</code>	

```

131 def search_from(maze, start_row, start_col):
132     # Schildkröte auf die Startposition setzen
133     maze.update_position(start_row, start_col, BLANK)
134     # Richtung festlegen
135     heading = "up"
136     print(heading)
137     # Schildkröte einstellen
138     maze.init_search(heading)
139
140     # Drehungen zählen
141     turn_count = 0
142     # Drehungs-Level berechnen: Linksdrehung +1, Rechtsdrehung -1
143     turn_level = 0
144

```

```
145 # Wiederhole bis Exit
146 while maze.is_exit(start_row, start_col) == False:
147
148     # solange vorne frei ist
149     print("turn_level =", turn_level)
150     while maze.look_forward(start_row, start_col, heading) == BLANK:
151         start_row, start_col = maze.one_step(start_row, start_col, heading)
152         maze.update_position(start_row, start_col)
153     # drehen, damit rechte Hand an der Wand ist
154     heading = maze.turn_left(heading)
155     turn_count += 1
156     turn_level += 1
157
158     # wenn Exit, dann Schleife abbrechen
159     if maze.is_exit(start_row, start_col) == True:
160         break
161
162     # solange der Drehungs-Level ungleich Null ist
163     while turn_level != 0:
164
165         # solange vorne frei und die Wand rechts ist
166         print("turn_level =", turn_level)
167         while maze.look_forward(start_row, start_col, heading) == BLANK\
168             and maze.look_right(start_row, start_col, heading) == OBSTACLE:
169             start_row, start_col = maze.one_step(start_row, start_col, heading)
170             maze.update_position(start_row, start_col)
171
172         # wenn Exit, dann Schleife abbrechen
173         if maze.is_exit(start_row, start_col) == True:
174             break
175
176         # wenn die Wand nicht mehr rechts ist
177         elif maze.look_right(start_row, start_col, heading) == BLANK:
178             # drehen und 1 Schritt vorwärts, damit rechte Hand an der Wand ist
179             heading = maze.turn_right(heading)
180             turn_count += 1
181             turn_level -= 1
182             start_row, start_col = maze.one_step(start_row, start_col, heading)
183             maze.update_position(start_row, start_col)
184
185         # wenn vorne eine Wand ist
186         elif maze.look_forward(start_row, start_col, heading) == OBSTACLE:
187             # drehen, damit rechte Hand an der Wand ist
188             heading = maze.turn_left(heading)
189             turn_count += 1
190             turn_level += 1
191
192     # Ende äußeren while-Schleife
193     print("Exit found!")
194     print(turn_count, "Drehungen")
```

Auto im Gegenverkehr: Auto steuern

Wir arbeiten mit der Bibliothek "Pygame" für Computerspiele

Wir steuern das Auto nach links – nach rechts – vorwärts – rückwärts -

`Spieler_Klasse_Auto_steuern.py`

Das Programm bringt ein Fenster mit einem blauen Auto auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Pfeiltasten der Tastatur. Warum verlässt das Auto das Fenster nicht?	
Ersetze das blaue Auto durch das größere gelbe Auto. Verlässt das gelbe Auto das Fenster?	

```

1 | # Programm erzeugt ein fenster mit einem Auto
2 | # - Das Auto wird mit den Pfeil-Tasten gesteuert
3 | # Bibliotheken importieren
4 | import pygame, sys
5 | # alle Module initialisieren
6 | pygame.init()
7 | # Frames per second festlegen
8 | FPS = 60
9 | # Clock objekt erzeugen
10 | FramePerSec = pygame.time.Clock()
11 | # Breite und Höhe des Fensters festlegen
12 | SCREEN_WIDTH = 400
13 | SCREEN_HEIGHT = 600
14 | # Farbe definieren
15 | WHITE = (255, 255, 255)
16 | # Grafik-Fenster erzeugen
17 | screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
18 | screen.fill(WHITE)
19 | # Spieler-Klasse
20 | class Player(pygame.sprite.Sprite):
21 |     """Klasse für den Spieler"""
22 |     def __init__(self):
23 |         super().__init__()
24 |         # Bild laden
25 |         self.image = pygame.image.load("blue_car.png")
26 |         # Rechteck in der Größe des Bildes erzeugen
27 |         self.rect = self.image.get_rect()
28 |         # Anfangsposition definieren
29 |         self.rect.center = (160, 520)
30 |     # Player bewegen
31 |     def move(self):
32 |         # Zustand aller Tasten abfragen
33 |         pressed_keys = pygame.key.get_pressed()
34 |         # Wenn noch im Fenster: Player mit Taste steuern
35 |         if self.rect.left > 0:
36 |             if pressed_keys[pygame.K_LEFT]:
37 |                 self.rect.move_ip(-5, 0)
38 |         if self.rect.right < SCREEN_WIDTH:
39 |             if pressed_keys[pygame.K_RIGHT]:
40 |                 self.rect.move_ip(5, 0)
41 |         if self.rect.top > 0:
42 |             if pressed_keys[pygame.K_UP]:
43 |                 self.rect.move_ip(0, -5)
44 |         if self.rect.bottom < SCREEN_HEIGHT:
45 |             if pressed_keys[pygame.K_DOWN]:
46 |                 self.rect.move_ip(0, 5)

```

```
47 # Instanz erzeugen
48 P1 = Player()
49 #Game Loop
50 while True:
51     # Ereignisse prüfen
52     for event in pygame.event.get():
53         if event.type == pygame.QUIT:
54             # pygame Fenster schließen
55             pygame.quit()
56             # Python Skript beenden
57             sys.exit()
58     # Player bewegen
59     P1.move()
60     # Grafik-Fenster auswischen
61     screen.fill(WHITE)
62     # Player zeichnen
63     # Blocktransfer: source = P1.image, destination = P1.rect
64     screen.blit(P1.image, P1.rect)
65     # Grafik-Fenster aktualisieren
66     pygame.display.update()
67     # Frames per second begrenzen
68     FramePerSec.tick(FPS)
```

Auto im Gegenverkehr: Gegenverkehr kommt

Ein Auto kommt entgegen – Spieler_Klasse_Gegner_Klasse_Gegenverkehr.py

Das Programm bringt ein Fenster mit einem blauen Auto *und einem entgegenkommenden roten Auto* auf den Bildschirm. Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Was passiert bei einem Zusammenstoß der beiden Autos?	
Verdopple die Geschwindigkeit des roten Autos. Die Geschwindigkeit des blauen Autos soll dabei unverändert bleiben.	

```

43 # Gegner-Klasse
44 class Enemy(pygame.sprite.Sprite):
45     def __init__(self):
46         super().__init__()
47         # Bild laden
48         self.image = pygame.image.load("red_car.png")
49         # Rechteck in der Größe des Bildes erzeugen
50         self.rect = self.image.get_rect()
51         # zufällige Anfangsposition
52         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
53     # Enemy bewegen
54     def move(self):
55         self.rect.move_ip(0,10)
56         # Wenn im Fenster unten angekommen
57         if (self.rect.bottom > SCREEN_HEIGHT):
58             # oben im Fenster beginnen
59             self.rect.top = 0
60             # zufällige Anfangsposition
61             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
62 # Instanzen erzeugen
63 P1 = Player()
64 E1 = Enemy()
65 #Game Loop
66 while True:
67     # Ereignisse prüfen
68     for event in pygame.event.get():
69         if event.type == pygame.QUIT:
70             # pygame Fenster schließen
71             pygame.quit()
72             # Python Skript beenden
73             sys.exit()
74     # Player bewegen
75     P1.move()
76     # Enemy bewegen
77     E1.move()
78     # Grafik-Fenster auswischen
79     screen.fill(WHITE)
80     # Player zeichnen
81     # Blocktransfer: source = P1.image, destination = P1.rect
82     screen.blit(P1.image, P1.rect)
83     # Enemy zeichnen
84     # Blocktransfer: source = E1.image, destination = E1.rect
85     screen.blit(E1.image, E1.rect)
86     # Grafik-Fenster aktualisieren
87     pygame.display.update()
88     # Frames per second begrenzen
89     FramePerSec.tick(FPS)

```

Auto im Gegenverkehr: Zusammenstoß melden

Zusammenstoß melden - Spieler_Klasse_Gegner_Klasse_Zusammenstoss.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf den Bildschirm. *Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und das Programm endet.* Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Was passiert bei einem Zusammenstoß der beiden Autos?	
Erzeuge eine weitere Instanz der Gegner-Klasse. Erweitere die Gruppen <code>enemies</code> und <code>all_sprites</code> , sodass zwei rote Autos entgegenkommen.	

```

19 # Farbe definieren
20 WHITE = (255, 255, 255)
21 BLACK = (0, 0, 0)
22 RED = (255, 0, 0)
23 # Schriften definieren
24 font_big = pygame.font.SysFont("Verdana", 40)
25 font_small = pygame.font.SysFont("Verdana", 20)
26 # Text definieren
27 meldung = font_big.render("Zusammenstoß", True, RED)

53 # Gegner-Klasse
54 class Enemy(pygame.sprite.Sprite):
55     def __init__(self):
56         super().__init__()
57         # Bild laden
58         self.image = pygame.image.load("red_car.png")
59         # Rechteck in der Größe des Bildes erzeugen
60         self.rect = self.image.get_rect()
61         # zufällige Anfangsposition
62         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
63         # score definieren
64         self.score = 0
65     # Enemy bewegen
66     def move(self):
67         self.rect.move_ip(0,10)
68         # Wenn im Fenster unten angekommen
69         if (self.rect.bottom > SCREEN_HEIGHT):
70             # score erhöhen
71             self.score += 1
72             # oben im Fenster beginnen
73             self.rect.top = 0
74             # zufällige Anfangsposition
75             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
76 # Instanzen erzeugen
77 P1 = Player()
78 E1 = Enemy()
79 # Gegner-Gruppe erzeugen
80 enemies = pygame.sprite.Group()
81 enemies.add(E1)
82 # Gruppe mit Spieler und Gegner
83 all_sprites = pygame.sprite.Group()
84 all_sprites.add(P1)
85 all_sprites.add(E1)
86 # Funktion beendet das Spiel
87 def end_of_game():
88     # pygame Fenster schließen
89     pygame.quit()
90     # Python Skript beenden
91     sys.exit()

```

```
92 #Game Loop
93 while True:
94     # Ereignisse prüfen
95     for event in pygame.event.get():
96         if event.type == pygame.QUIT:
97             # Spiel-Ende
98             end_of_game()
99     # Grafik-Fenster auswischen
100    screen.fill(WHITE)
101    # score zeichnen
102    my_score = font_small.render(str(E1.score), True, BLACK)
103    screen.blit(my_score, (10,10))
104    # Spieler und Gegner bewegen und zeichnen
105    for entity in all_sprites:
106        # Spieler und Gegner bewegen
107        entity.move()
108        # Blocktransfer: source = entity.image, destination = entity.rect
109        screen.blit(entity.image, entity.rect)
110    # Zusammenstoß entdecken
111    if pygame.sprite.spritecollideany(P1, enemies):
112        # Zusammenstoß melden
113        screen.blit(meldung, (30,250))
114        # Grafik-Fenster aktualisieren
115        pygame.display.update()
116        # Warten
117        time.sleep(5)
118        # Spiel-Ende
119        end_of_game()
120    # Grafik-Fenster aktualisieren
121    pygame.display.update()
122    # Frames per second begrenzen
123    FramePerSec.tick(FPS)
```


Auto im Gegenverkehr: Gegenverkehr wird schneller

Gegenverkehr wird schneller werden - Spieler_Klasse_Gegner_Klasse_schneller.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf den Bildschirm. Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und das Programm endet. *Das rote Auto wird mit jeder Sekunde schneller.* Starte das Programm. Ändere das Programm und starte es erneut.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Mach 4 Läufe und notiere die Scores.	
Nun soll das Auto jede Sekunde die Geschwindigkeit wechseln. Die Geschwindigkeit ist ein Zufallswert zwischen 5 und 20. Mach 4 Läufe und notiere die Scores.	

```

54 # Gegner-Klasse
55 class Enemy(pygame.sprite.Sprite):
56     def __init__(self):
57         super().__init__()
58         # Bild laden
59         self.image = pygame.image.load("red_car.png")
60         # Rechteck in der Größe des Bildes erzeugen
61         self.rect = self.image.get_rect()
62         # zufällige Anfangsposition
63         self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0)
64         # score definieren
65         self.score = 0
66         # speed definieren
67         self.speed = 5
68     # Enemy bewegen
69     def move(self):
70         self.rect.move_ip(0, self.speed)
71         # Wenn im Fenster unten angekommen
72         if (self.rect.bottom > SCREEN_HEIGHT):
73             # score erhöhen
74             self.score += 1
75             # oben im Fenster beginnen
76             self.rect.top = 0
77             # zufällige Anfangsposition
78             self.rect.center = (random.randint(40, SCREEN_WIDTH - 40), 0)
79
80
81 # User-Ereignis INC_SPEED: eindeutige ID definieren
82 INC_SPEED = pygame.USEREVENT + 1
83 # User-Ereignis alle 1000 Millisekunden erscheinen lassen
84 pygame.time.set_timer(INC_SPEED, 1000)
85
86
87 #Game Loop
88 while True:
89     # Ereignisse prüfen
90     for event in pygame.event.get():
91         if event.type == INC_SPEED:
92             # Gegner schneller machen
93             E1.speed += 1
94         if event.type == pygame.QUIT:
95             # Spiel-Ende
96             end_of_game()

```

Auto im Gegenverkehr: Straße und Sound

Straße im Hintergrund, Sound beim Zusammenstoß – Traffic_Game.py

Das Programm bringt ein Fenster mit einem blauen Auto und einem entgegenkommenden roten Auto auf einer Straße auf den Bildschirm. Jedes erfolgreiche Ausweichen des Spielers wird gezählt. Bei einem Zusammenstoß gibt es eine Meldung und einen Sound und das Programm endet. Das rote Auto wird mit jeder Sekunde schneller. Starte das Programm.

Aufgabe	Ergebnis
Steuere das Auto mit den Links- und Rechts-Pfeiltasten der Tastatur. Wer erreicht den höchsten Score?	

```
26| # Bild für den Hintergrund laden
27| background = pygame.image.load("animated_street.png")

113|     # Straße zeichnen
114|     screen.blit(background, (0,0))

126|         # Sound abspielen
127|         pygame.mixer.Sound('crash.wav').play()
```

Ein Dinosaurier überquert die Straße

Ein Dinosaurier überquert die Straße. Baue das Programm Traffic_Game.py um.

Baue das Programm Traffic_Game.py Schritt für Schritt um:

1. Kein blaues Auto – Dinosaurier. Kein Zusammenstoß – das Auto bremst rechtzeitig
2. Dinosaurier geht immer mit dem Kopf voran.
3. Auf dem Gehweg ist der Dinosaurier sicher.
4. Jedes erfolgreiche Überqueren der Straße wird gezählt.

Teste das Ergebnis nach jedem Schritt.

Nr.	Aufgabe	Ergebnis
1	Lösche die Zeilen 3 bis 8. Schreibe in Zeile 2: Ein Dinosaurier überquert die Straße Ersetze die Meldung "Zusammenstoß" durch "Achtung!" Ersetze "blue_car.png" durch "dinosaur_LR.png" Ersetze "crash.wav" durch "car_screech.wav"	
2	Spieler-Klasse: Lade 1 Dino-Bild für links → rechts und 1 Dino-Bild für rechts → links: self.image_LR = pygame.image.load("dinosaur_LR.png") self.image_RL = pygame.image.load("dinosaur_RL.png") Definiere das Anfangsbild: self.image = self.image_LR Wechsle die Bilder abhängig von pressed_keys[:] self.image = self.image_RL und self.image = self.image_LR Der Dinosaurier muss schneller laufen, um die Straße zu überqueren: self.rect.move_ip(-10, 0) und self.rect.move_ip(10, 0)	
3	Setze SCREEN_WIDTH = 600 Ersetze "animated_street.png" durch "animated_street_w_sidewalk.png" Zu Beginn steht der Dinosaurier auf dem Gehweg. Ersetze in der Spieler-Klasse self.rect.center = (160, 520) durch self.rect.center = (60, 520) Das rote Auto darf nicht auf dem Gehweg fahren. Ersetze in den Methoden __init__() und move() der Gegner-Klasse: self.rect.center = (random.randint(40, SCREEN_WIDTH-40), 0) durch: self.rect.center = (random.randint(150, SCREEN_WIDTH-150), 0)	
4	Das Auto-Spiel berechnet den Score in der Gegner-Klasse. Unser Dinosaurier-Spiel berechnet den Score in der <u>Spieler-Klasse</u> . In der Methode __init__() der Spieler-Klasse werden neue Eigenschaften definiert: self.direction = "right" self.score = 0 In der Methode move() der Spieler-Klasse werden zwei neue Bedingungen geprüft: # Wenn die Straße überquert wurde if (self.direction == "right") and (self.rect.left > SCREEN_WIDTH-120): self.score += 1 self.direction = "left" if (self.direction == "left") and (self.rect.right < 120): self.score += 1 self.direction = "right" Die Variable my_score() wird aus P1.score berechnet: my_score = font_small.render(str(P1.score), True, BLACK)	

MicroPython und ESP32: Hardware und Software

Wir verwenden den SunFounder ESP32 Starter Kit

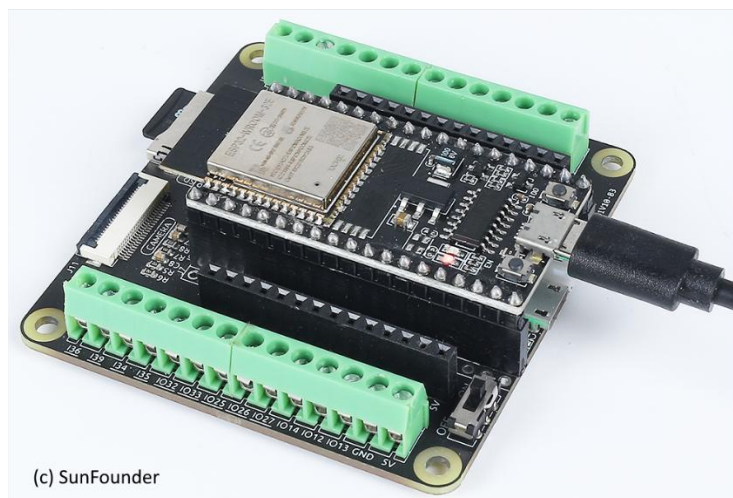
<https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/>

Über MicroPython:

- MicroPython ist eine schlanke Implementierung von Python 3.
- MicroPython ist optimiert und läuft auf Microcontrollern.
- MicroPython präsentiert dem Benutzer die REPL >>>.
- MicroPython hat Python-Bibliotheken zum Zugriff auf die Hardware.
- Die MicroPython Community pflegt das Open-Source Projekt.

Über den Mikrocontroller ESP32:

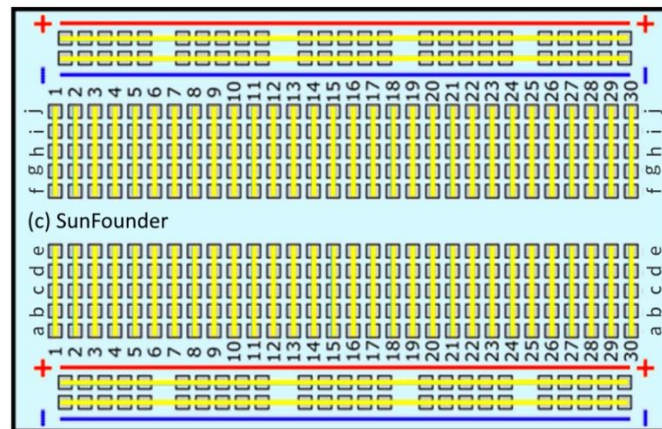
- Der ESP32 ist ein leistungsstarker, vielseitiger Mikrocontroller.
- Wir programmieren den ESP32 mit der IDE **Thonny** auf dem PC.
- Der ESP32 steckt in einem **Erweiterungsboard** und wird über ein USB-Kabel an den PC angeschlossen.
- Wir haben MicroPython bereits mit Hilfe der IDE Thonny auf dem ESP32 installiert.
- Im Internet werden Code und Libs bereitgestellt. Wir haben die Libs bereits auf dem ESP32 installiert.



Erweiterungsboard mit ESP32

Über das Steckbrett:

- Das Steckbrett ist eine rechteckige Kunststoffplatte mit vielen kleinen Löchern.
- Diese Löcher ermöglichen es dir, elektronische Bauteile einzufügen, um Schaltungen zu bauen.
- Die Löcher des Steckbretts sind entlang der gelben Linien miteinander verbunden.



Steckbrett

Vorsichtsmaßnahmen:

- **Zuerst** das ESP32 USB Kabel vom PC **trennen**, dann Erweiterungsboard, Steckbrett und Bauteile verdrahten.
- Die GPIO-Pins des ESP32 sind nicht 5 V tolerant. **Maximal 3,3 V** sind erlaubt.
- **Kein Kurzschluss!** Bei einem Kurzschluss fließt ein hoher Strom, der GPIOs, Dioden und Transistoren zerstört.

Allgemeine Vorgehensweise:

- Zuerst die Verdrahtung prüfen. **Wenn OK**, das ESP32 USB Kabel an den PC **anschießen**.
- Programmcode mit **Thonny** öffnen und mit **Run** ausführen.
- Laufenden Programmcode mit **Stop** abbrechen.
- Wenn es nicht funktioniert: **Zuerst** das ESP32 USB Kabel vom PC **trennen**, dann die Verdrahtung korrigieren.

MicroPython: Displays, Töne, Aktoren



Blinkende LED - 2.1_hello_led.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_led.html und folge den Anweisungen.



Zeichenanzeige - 2.6_liquid_crystal_display.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_lcd.html und folge den Anweisungen.



Beep - 3.1_beep.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_ac_buz.html und folge den Anweisungen.



Eigener Klang - 3.2_custom_tone.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_pa_buz.html und folge den Anweisungen.

MicroPython: Sensoren



Lesen des Tastenwerts - 5.1_read_button_value.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_button.html und folge den Anweisungen.



Hinderniserkennung - 5.3_avoid.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_avoid.html und folge den Anweisungen.



Distanzmessung - 5.12_ultrasonic.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_ultrasonic.html und folge den Anweisungen.

MicroPython: Einparkhilfe



Einparkhilfe - 6.4_reversing_aid.py

Öffne die Internet-Seite https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/basic_projects/py_reversing_aid.html und folge den Anweisungen.

Wenn das Programm nicht funktioniert, funktioniert möglicherweise das Multithreading nicht.

Hier steht eine wichtige Information zum Multithreading-Modul:

<https://docs.micropython.org/en/latest/library/thread.html#module-thread>

"This module is highly experimental and its API is not yet fully settled and not yet described in this documentation."

Teste deshalb das Programm 6.4a_reversing_aid.py. Das Programm arbeitet ohne Multithreading.

Ende

Quellen

Quelle	Thema
https://docs.python.org/3/	Python Grundlagen
https://www.python-lernen.de/	Python Grundlagen
Felleisen et al. (2013), Realm of Racket, No Starch Press, San Francisco	Spiel: Computer errät die Zahl
https://www.deinprogramm.de/	Mantras, Konstruktionsanleitung für Funktionen
https://matplotlib.org/stable/tutorials/pyplot.html	Bibliothek matplotlib
https://docs.python.org/3/library/turtle.html	Bibliothek turtle
https://runestone.academy/ns/books/published/pythonds3/Recursion/ExploringaMaze.html	Irrgarten und Turtle
https://gist.github.com/maxrothman/92eb8470408a047b1f6815a4a444d727	Irrgarten lösen
https://algo.rwth-aachen.de/~algorithmus/algo6.php	Pledge Algorithmus
https://www.pygame.org/docs/index.html	Bibliothek pygame
https://coderslegacy.com/python/python-pygame-tutorial/	Spiel: Traffic Game
https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/micropython/for_micropython_user.html	Für MicroPython Anwender
https://docs.sunfounder.com/projects/esp32-starter-kit/de/latest/components/component_list.html	Komponenten im Starter Kit